University of BRISTOL

LECTURE 3

Linear classification

Foundations of Artificial Intelligence

SEMTM0051 · 2026/27

Lecturer: Dr Roberto Colomboni



Today's lecture

Binary classification

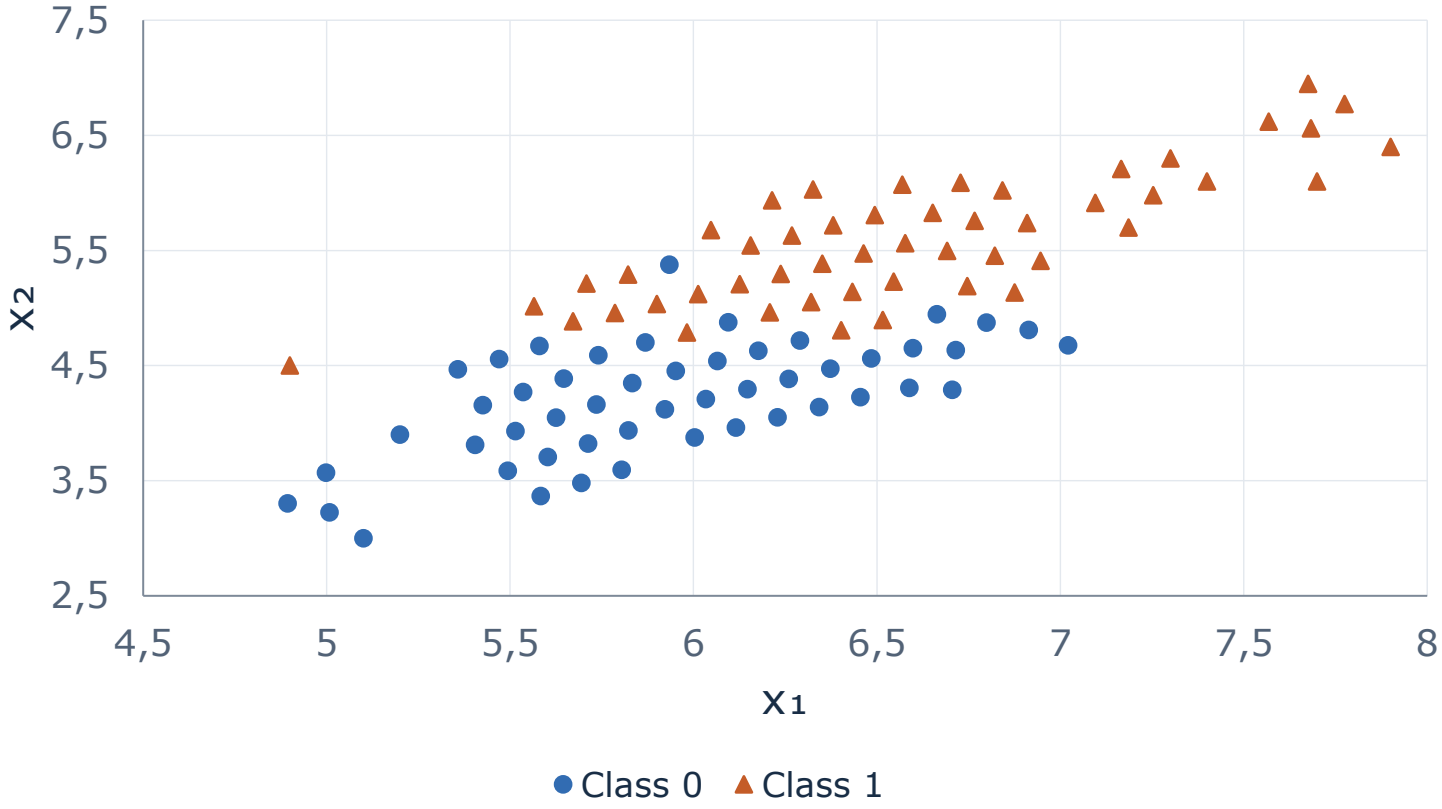
Classification error and logistic loss

Gradient descent

More than two classes

Binary classification

A synthetic dataset adapted from Iris:
100 observations $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_{100}, y_{100})$, with two input features and two classes.



$\mathbf{x}_i = (x_{i1}, x_{i2}), \quad y_i \in \{0,1\}$

x_1: first input feature

x_2: second input feature

Class 0: 50 observations

Class 1: 50 observations

Learn a rule to predict the class of an input.

Dataset and instructions in the folder: Week 3 - Examples

Measuring classification error

A regression predictor $f\colon \mathbb{R}^d \to \mathbb{R}$ returns a number

Regression

$$\ell_{\text{sq}}(\hat{y}, y) = (\hat{y} - y)^2$$

Empirical squared loss

$$\sum_{i=1}^{N} \ell_{\text{sq}}(f(\mathbf{x}_i), y_i)$$

Larger numerical errors receive larger penalties

A classifier $h\colon \mathbb{R}^d \to \{0,1\}$ returns a class label.

Classification

$$\ell_{0,1}(\hat{y}, y) = \begin{cases} 0, & \hat{y} = y \\ 1, & \hat{y} \neq y \end{cases}$$

Number of mistakes / Empirical 0-1 loss

$$\sum_{i=1}^{N} \ell_{0,1}(h(\mathbf{x}_i), y_i)$$

All mistakes are penalized the same way

Predicting a probability first

A label gives a decision.

A probability also records how strongly the model favours that label.

Probabilistic predictor

$$p : \mathbb{R}^d \longrightarrow [0,1]$$

Probability of class 1

$$p(\mathbf{x})$$

Probability of class 0

$$1 - p(\mathbf{x})$$

Example: if $p(\mathbf{x}) = 0.51$ we think that there is

- a 0.51 probability that the correct class of input $\mathbf{x}$ is 1
- a 0.49 probability that the correct class of input $\mathbf{x}$ is 0

If we must choose one label, which should we predict?

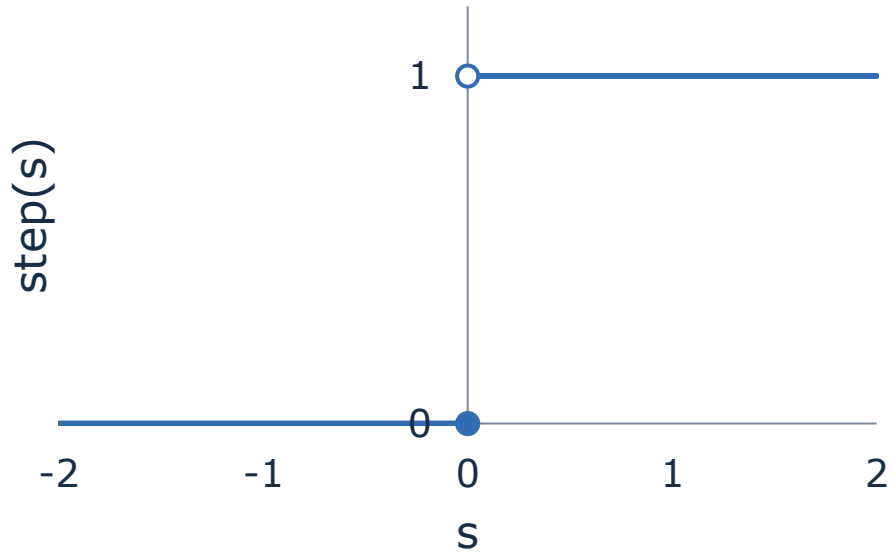
The plug-in classification rule

Choose the label with the larger probability. At a tie, we choose class 0.

$$h_p(\mathbf{x}) = \begin{cases} 0, & p(\mathbf{x}) \leq 1/2 \\ 1, & p(\mathbf{x}) > 1/2 \end{cases}$$

Heaviside step function

$$\text{step}(s) := \begin{cases} 0, & s \leq 0, \\ 1, & s > 0. \end{cases}$$



$$h_p(\mathbf{x}) = \text{step}\left(p(\mathbf{x}) - \frac{1}{2}\right)$$

How do we choose the probability function?

How do we build probability predictions?

Regression suggests using an intercept and one weight for each input feature.

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = b + \mathbf{x}\mathbf{w} = b + \sum_{j=1}^{d} x_j w_j$$

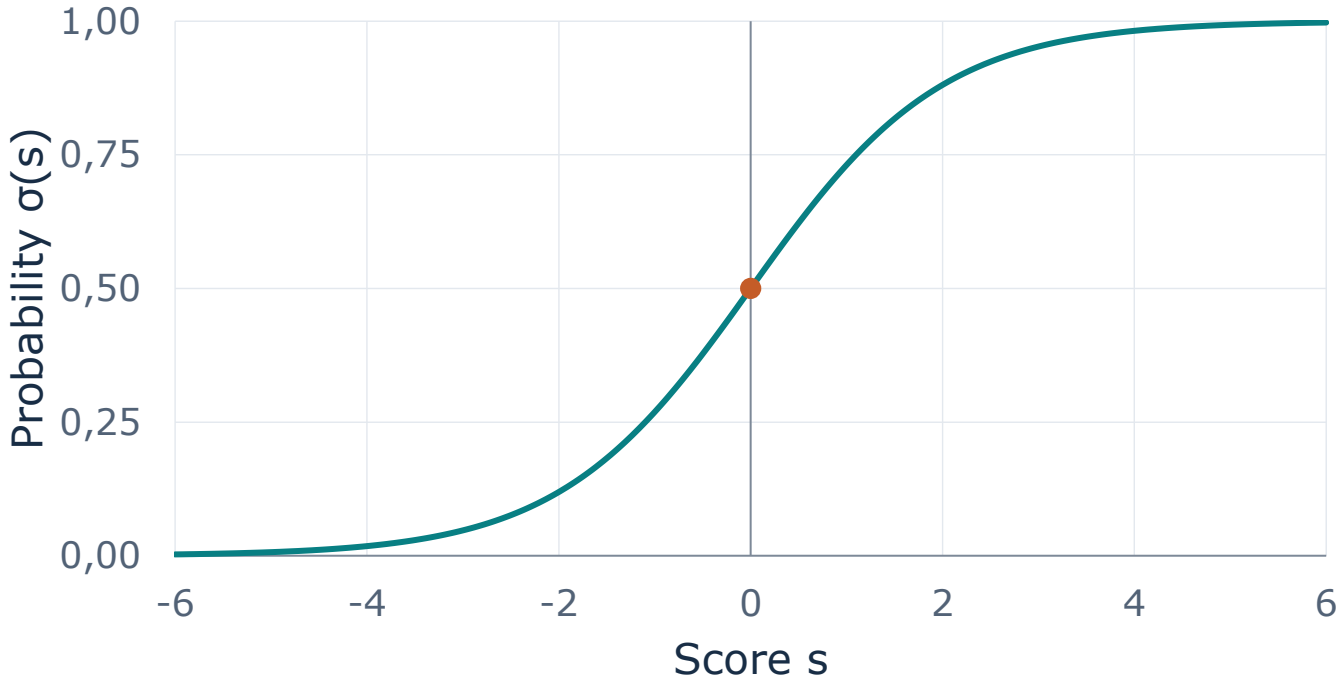
$$b \in \mathbb{R}, \qquad \mathbf{w} = (w_1, \ldots, w_d)^\top \in \mathbb{R}^{d \times 1}$$

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ \mathbf{w} \end{pmatrix} = (b, w_1, \ldots, w_d)^\top \in \mathbb{R}^{(d+1) \times 1}$$

However, $f_{\boldsymbol{\theta}}(\mathbf{x})$ can be any real number!
Hence, it cannot serve directly as a probability.
Rather, $f_{\boldsymbol{\theta}}(\mathbf{x})$ is a score (also called a logit)
that we have to turn into a probability.

The sigmoid function

A smooth, increasing function turns each real score into a number between zero and one.



$$\sigma(s) := \frac{1}{1+e^{-s}} = \frac{e^{s}}{1+e^{s}}$$

$$0 < \sigma(s) < 1$$

$$\sigma(0) = \frac{1}{2}$$

Probabilities, labels and the boundary

Using the sigmoid σ we can turn scores into probabilities:

$$p_{\boldsymbol{\theta}}(\mathbf{x}) \coloneqq \sigma\big(f_{\boldsymbol{\theta}}(\mathbf{x})\big) = \sigma(b + \mathbf{x}\mathbf{w})$$

The corresponding plug-in classifier $h_{p_{\boldsymbol{\theta}}}$ satisfies

$$h_{p_{\boldsymbol{\theta}}}(\mathbf{x}) = 1 \quad \Leftrightarrow \quad p_{\boldsymbol{\theta}}(\mathbf{x}) > \frac{1}{2} \quad \Leftrightarrow \quad f_{\boldsymbol{\theta}}(\mathbf{x}) > 0 \quad \Leftrightarrow \quad b + \mathbf{x}\mathbf{w} > 0$$

Decision boundary: the set of $\mathbf{x} \in \mathbb{R}^d$ such that $b + \mathbf{x}\mathbf{w} = 0$

The decision boundary is a hyperplane with d features,
provided the weight vector $\mathbf{w}$ is nonzero.

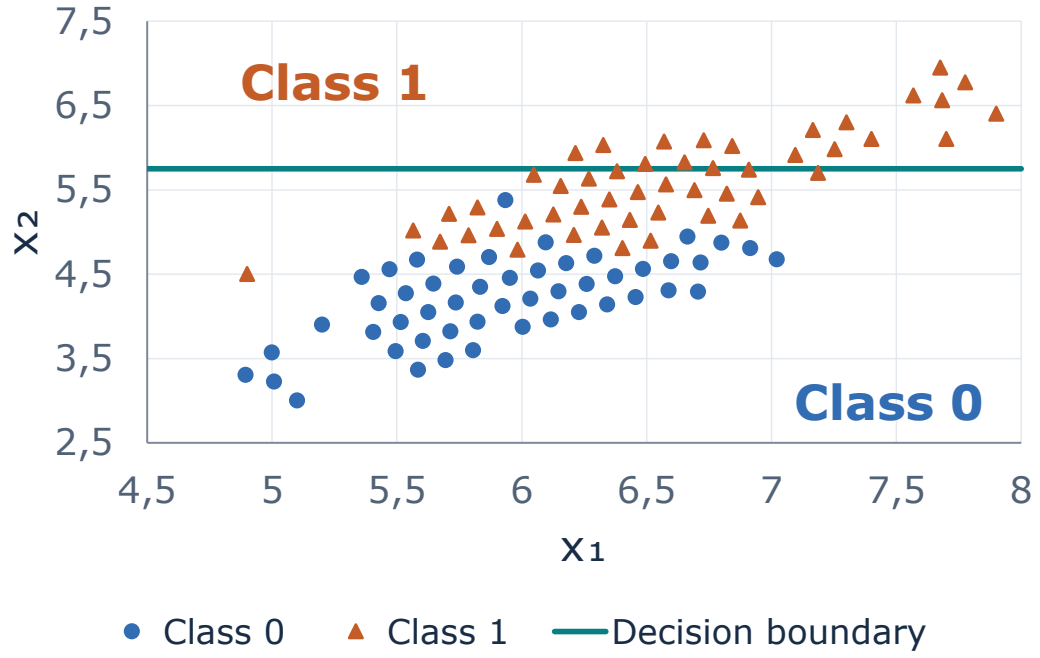
If $d = 1$, $b + x_1 w_1 = 0$ determines a point of the Euclidean line

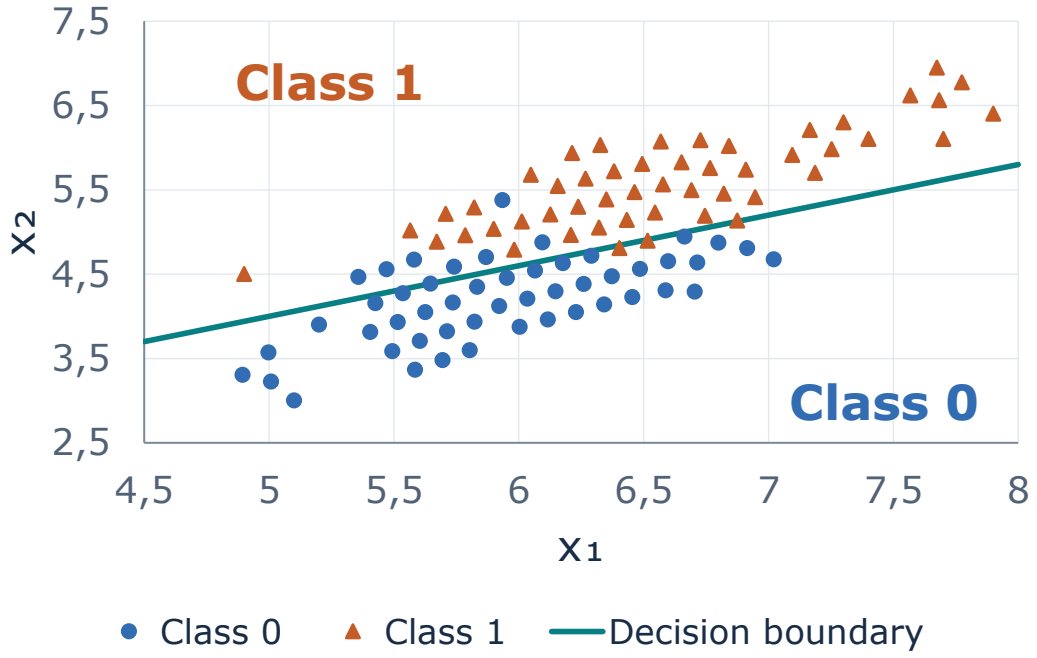
If $d = 2$, $b + x_1 w_1 + x_2 w_2 = 0$ determines a straight line of the Euclidean plane

If $d = 3$, $b + x_1 w_1 + x_2 w_2 + x_3 w_3 = 0$ determines a plane of the Euclidean space

Comparing two candidate boundaries

Which of these two classifiers is better?
Count the number of mistakes!





$\boldsymbol{\theta} = (-5.75, 0, 1)^\top, \quad x_2 = 5.75$ $\boldsymbol{\theta} = (-1, -0.6, 1)^\top, \quad x_2 = 0.6x_1 + 1$

The classifier on the right makes fewer mistakes on these observations

Today's lecture

Binary classification

Classification error and logistic loss

Gradient descent

More than two classes

Finding the parameters

We want the smallest mistake count.

Finding the parameter $\boldsymbol{\theta}^*$ that minimizes the empirical 0-1 loss:

$$\boldsymbol{\theta} \longmapsto \mathcal{E}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell_{0,1}\left(h_{p_{\boldsymbol{\theta}}}(\mathbf{x}_i), y_i\right)$$

is a combinatorial problem.

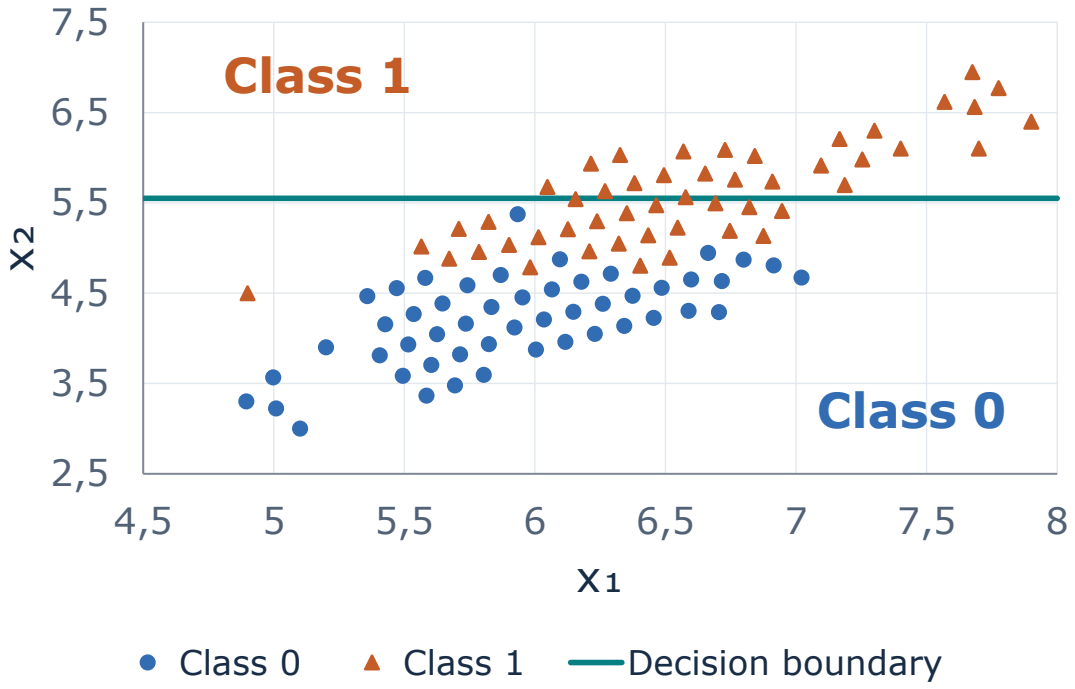
Can we compute an approximate solution via gradient descent as in regression?

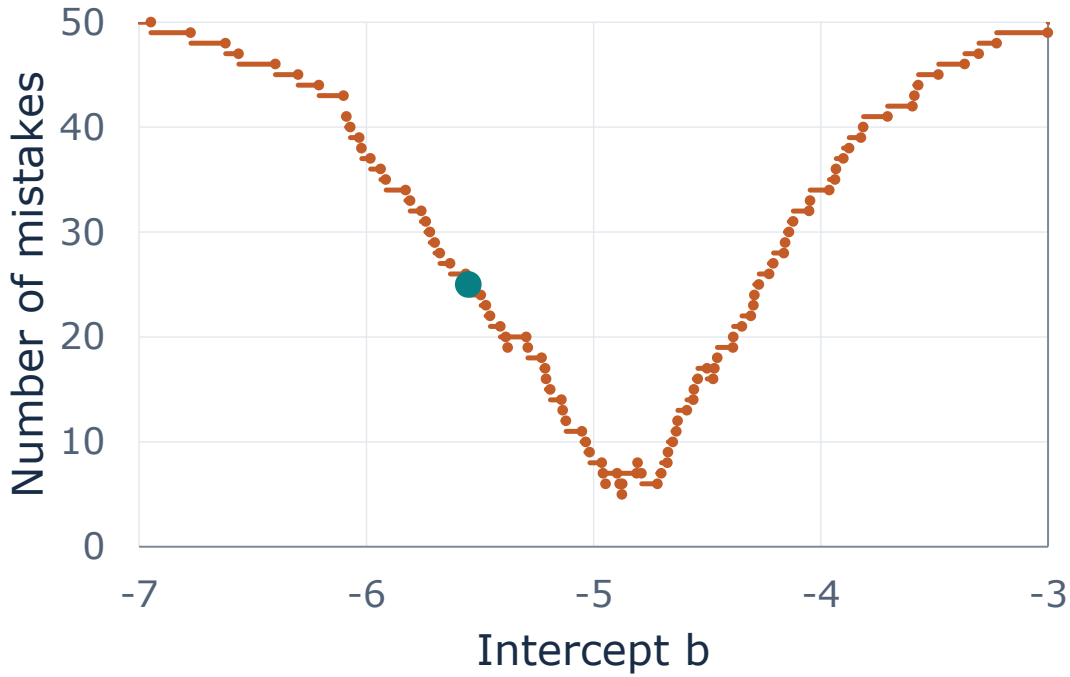
$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{E}\left(\boldsymbol{\theta}^{(t)}\right), \qquad \eta > 0$$

What happens to the mistake count when we move the boundary a little?

Moving the boundary and counting mistakes

Fix the weights and vary only the intercept. The line moves parallel to itself.





$$b = -5.55, \qquad w_1 = 0, \quad w_2 = 1$$

The count is locally constant and changes only when the line crosses an observed point.

Flat regions give no direction

If every observed score is nonzero
small parameter changes preserve the signs of the scores
and hence all predicted labels stay the same.

$$\mathcal{E}(\boldsymbol{\theta} + \boldsymbol{\Delta}) = \mathcal{E}(\boldsymbol{\theta}) \quad \text{for sufficiently small } \boldsymbol{\Delta}$$

The gradient of a locally constant function is zero!

$$\nabla \mathcal{E}(\boldsymbol{\theta}) = \mathbf{0}$$

Hence

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{E}\left(\boldsymbol{\theta}^{(t)}\right) = \boldsymbol{\theta}^{(t)}$$

Gradient descent will not move!!!

What does the mistake count miss?

Suppose the observed class is 0. Both predictions below choose class 1.

	Prediction A	Prediction B
Probability of class 1	0.51	0.90
Predicted class	1	1
Zero–one loss	1	1
Probability of the observed class	**0.49**	**0.10**

At 0.51, the model is almost guessing: we want a milder penalty.

At 0.90, it is confidently wrong: we want a larger penalty.

Rather than scoring only the predicted label,
we penalise assigning low probability to the observed class.

Logistic loss: a surrogate for fitting probabilities

For $0 < p < 1$ and $y \in \{0, 1\}$, take the negative logarithm of the probability assigned to the observed class.

$$\ell_{\log}(p, y) = -y\log(p) - (1 - y)\log(1 - p) = \begin{cases} -\log(p), & y = 1, \\ -\log(1 - p), & y = 0. \end{cases}$$

The logistic loss makes us pay the negative logarithm of the probability we assign to the correct label.

Wrong, but uncertain

$$\ell_{\log}(0.51, 0) = -\log(0.49) \approx 0.713$$

Confidently wrong

$$\ell_{\log}(0.90, 0) = -\log(0.10) \approx 2.303$$

Logistic loss is also called binary cross-entropy. We use natural logarithms.

The empirical logistic loss

The observations stay fixed.
The probability predictions depend on the parameters.

$$p_{\boldsymbol{\theta}}(\mathbf{x}_i) = \sigma(b + \mathbf{x}_i \mathbf{w})$$

$$\mathcal{L}_{\log}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)$$

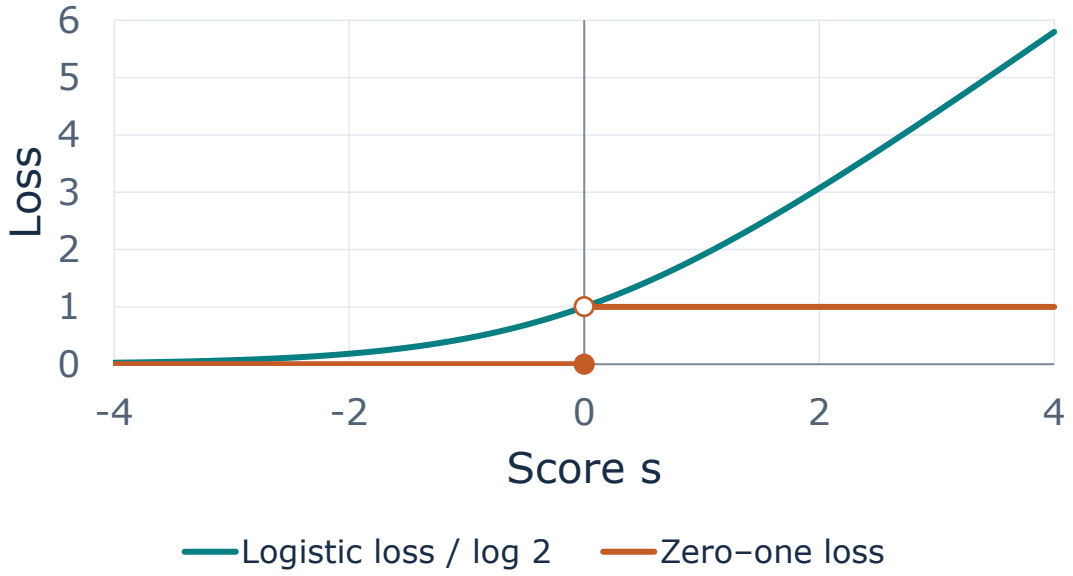
We fit this smooth surrogate loss using gradient descent,
while the mistake count $\mathcal{E}(\boldsymbol{\theta})$ remains our classification measure.

Comparing the two losses

A mistake gives the observed class probability at most $1/2$,

so its logistic loss is at least $-\log\left(\frac{1}{2}\right) = \log(2)$.

Observed label $y = 0$



Observed label $y = 1$



$$\ell_{0,1}\left(\text{step}\left(\sigma(s) - \frac{1}{2}\right), y\right) \leq \frac{\ell_{\log}(\sigma(s), y)}{\log(2)}$$

Bounding the number of mistakes

The inequality holds for each observation
We can therefore add it over the dataset

$$\mathcal{E}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell_{0,1}\left(h_{p_{\boldsymbol{\theta}}}(\mathbf{x}_i), y_i\right)$$

$$= \sum_{i=1}^{N} \ell_{0,1}\left(\text{step}\left(\sigma\left(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\right) - \frac{1}{2}\right), y_i\right) \leq \sum_{i=1}^{N} \frac{\ell_{\log}\left(\sigma\left(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\right), y_i\right)}{\log(2)}$$

$$= \frac{1}{\log(2)} \sum_{i=1}^{N} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \frac{1}{\log(2)} \cdot \mathcal{L}_{\log}(\boldsymbol{\theta})$$

A control over the empirical logistic loss
gives a control on the number of mistakes

Today's lecture

Binary classification

Classification error and logistic loss

Gradient descent

More than two classes

The design matrix and the parameters

As in Lecture 2, a first column of ones includes the intercept.

$$\mathbf{X} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1d} \\ \vdots & \vdots & & \vdots \\ 1 & x_{N1} & \cdots & x_{Nd} \end{pmatrix} \qquad \boldsymbol{\theta} = \begin{pmatrix} b \\ w_1 \\ \vdots \\ w_d \end{pmatrix}$$

$$\mathbf{X} \in \mathbb{R}^{N \times (d+1)}, \qquad \boldsymbol{\theta} \in \mathbb{R}^{(d+1) \times 1}$$

$$\theta_1 = b, \qquad \theta_{k+1} = w_k, \qquad k = 1, \ldots, d$$

The first coordinate is the intercept. The remaining coordinates are the weights.

The three functions

Fix the ith input/output pair $(\mathbf{x}_i, y_i)$ and vary the model parameters $\boldsymbol{\theta}$.

Define affine score corresponding to the ith input $\mathbf{x}_i$

$$F_i(\boldsymbol{\theta}) \coloneqq f_{\boldsymbol{\theta}}(\mathbf{x}_i) = b + \mathbf{x}_i \mathbf{w} = (1, \mathbf{x}_i)\boldsymbol{\theta} = \sum_{r=1}^{d+1} X_{ir}\theta_r$$

Recall the definition of the sigmoid function σ

$$\sigma(s) = \frac{1}{1 + e^{-s}} \qquad (s \in \mathbb{R})$$

Define the logistic loss corresponding to the ith output y_i

$$G_i(p) \coloneqq \ell_{\log}(p, y_i)$$

The chain rule for one observation

The loss for the ith observation is the composition of these three functions.

$$\ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = (G_i \circ \sigma \circ F_i)(\boldsymbol{\theta}) = G_i\left(\sigma\left(F_i(\boldsymbol{\theta})\right)\right)$$

To compute the partial derivatives with respect to θ_j we apply the chain rule to this composition.

$$\frac{\partial}{\partial \theta_j}\ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \frac{\partial}{\partial \theta_j}(G_i \circ \sigma \circ F_i)(\boldsymbol{\theta}) = G_i'\left(\sigma\left(F_i(\boldsymbol{\theta})\right)\right) \cdot \sigma'\left(F_i(\boldsymbol{\theta})\right) \cdot \frac{\partial F_i}{\partial \theta_j}(\boldsymbol{\theta})$$

$$= \frac{\partial \ell_{\log}}{\partial p}\left(\sigma\left(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\right), y_i\right) \cdot \sigma'\left(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\right) \cdot \frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)}{\partial \theta_j}$$

$$= \underbrace{\frac{\partial \ell_{\log}}{\partial p}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)}_{\text{first term}} \cdot \underbrace{\sigma'\left(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\right)}_{\text{second term}} \cdot \underbrace{\frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)}{\partial \theta_j}}_{\text{third term}}$$

First term: Differentiating the logistic loss

Hold the observed label fixed and vary the probability.

$$\ell_{\log}(p, y) = -y\log(p) - (1-y)\log(1-p)$$

$$\frac{\partial \ell_{\log}}{\partial p}(p, y) = -\frac{y}{p} + \frac{1-y}{1-p} = \frac{-y(1-p) + (1-y)p}{p(1-p)}$$
$$= \frac{-y + yp + p - yp}{p(1-p)} = \frac{p - y}{p(1-p)}$$

$$\boxed{\frac{\partial \ell_{\log}}{\partial p}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i}{p_{\boldsymbol{\theta}}(\mathbf{x}_i)\big(1 - p_{\boldsymbol{\theta}}(\mathbf{x}_i)\big)}}$$

Second term: Differentiating the sigmoid

Write the sigmoid as a composition and apply the chain rule again.

Inner function u

$$u(s) = 1 + e^{-s}, \qquad u'(s) = -e^{-s}$$

Outer function v

$$v(z) = \frac{1}{z}, \qquad v'(z) = -\frac{1}{z^2}$$

$$\sigma(s) = \frac{1}{1+e^{-s}} = v\big(u(s)\big)$$

$$\sigma'(s) = v'\big(u(s)\big)\,u'(s) = -\frac{1}{(1+e^{-s})^2}(-e^{-s}) = \frac{e^{-s}}{(1+e^{-s})^2}$$

$$= \frac{1}{1+e^{-s}} \cdot \frac{e^{-s}}{1+e^{-s}} = \frac{1}{1+e^{-s}} \cdot \left(1 - \frac{1}{1+e^{-s}}\right) = \sigma(s)\big(1-\sigma(s)\big)$$

$$\boxed{\sigma'\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\big) = \sigma\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\big)\Big(1 - \sigma\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\big)\Big) = p_{\boldsymbol{\theta}}(\mathbf{x}_i)\big(1 - p_{\boldsymbol{\theta}}(\mathbf{x}_i)\big)}$$

Third term: Differentiating the affine score

The inputs are fixed, so only the matching parameter contributes.

The affine score

$$f_{\boldsymbol{\theta}}(\mathbf{x}_i) = \sum_{r=1}^{d+1} X_{ir}\theta_r$$

The Kronecker delta

$$\frac{\partial \theta_r}{\partial \theta_j} = \delta_{rj} = \begin{cases} 1, & r = j, \\ 0, & r \neq j. \end{cases}$$

$$\frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \sum_{r=1}^{d+1} X_{ir}\theta_r = \sum_{r=1}^{d+1} X_{ir} \frac{\partial \theta_r}{\partial \theta_j} = \sum_{r=1}^{d+1} X_{ir}\delta_{rj} = X_{ij}$$

$$\boxed{\frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)}{\partial \theta_j} = X_{ij}} \qquad j = 1, \ldots, d+1$$

Putting the three terms together

Substitute the first, second and third terms into the chain rule.

$$\frac{\partial}{\partial \theta_j} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)$$

$$= \underbrace{\frac{\partial \ell_{\log}}{\partial p}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)}_{\text{first term}} \cdot \underbrace{\sigma'\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i)\big)}_{\text{second term}} \cdot \underbrace{\frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)}{\partial \theta_j}}_{\text{third term}}$$

$$= \underbrace{\frac{p_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i}{p_{\boldsymbol{\theta}}(\mathbf{x}_i)\big(1 - p_{\boldsymbol{\theta}}(\mathbf{x}_i)\big)}}_{\text{first term}} \cdot \underbrace{p_{\boldsymbol{\theta}}(\mathbf{x}_i)\big(1 - p_{\boldsymbol{\theta}}(\mathbf{x}_i)\big)}_{\text{second term}} \cdot \underbrace{X_{ij}}_{\text{third term}}$$

$$= (p_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i) X_{ij}$$

Empirical gradient and gradient descent

The empirical loss is a sum, so we add the observation derivatives.

$$\frac{\partial \mathcal{L}_{\log}}{\partial \theta_j}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \frac{\partial}{\partial \theta_j} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \sum_{i=1}^{N} X_{ij}(p_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i)$$

Sigmoid applied to each entry

$$\boldsymbol{\sigma}\big((s_1, \dots, s_N)^\top\big) \coloneqq \big(\sigma(s_1), \dots, \sigma(s_N)\big)^\top$$

One probability per observation

$$\mathbf{p}(\boldsymbol{\theta}) \coloneqq \big(p_{\boldsymbol{\theta}}(\mathbf{x}_1), \dots, p_{\boldsymbol{\theta}}(\mathbf{x}_N)\big)^\top$$

$$\mathbf{p}(\boldsymbol{\theta}) = \boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}), \qquad \mathbf{y} = (y_1, \dots, y_N)^\top$$

Gradient

$$\nabla \mathcal{L}_{\log}(\boldsymbol{\theta}) = \mathbf{X}^\top(\mathbf{p}(\boldsymbol{\theta}) - \mathbf{y}) = \mathbf{X}^\top(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y})$$

Fitting logistic regression

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}_{\log}\big(\boldsymbol{\theta}^{(t)}\big) = \boldsymbol{\theta}^{(t)} - \eta \mathbf{X}^\top\big(\boldsymbol{\sigma}\big(\mathbf{X}\boldsymbol{\theta}^{(t)}\big) - \mathbf{y}\big)$$

Stationary points and minimisers

Fact: the empirical logistic loss is convex in the parameters.
Therefore, any stationary point is a global minimiser.

However: a finite minimiser need not exist!!!
See the notes for a counterexample.

Also: Even when a finite minimiser exists,
the stationarity equations form a nonlinear system,
generally with no closed-form solution.

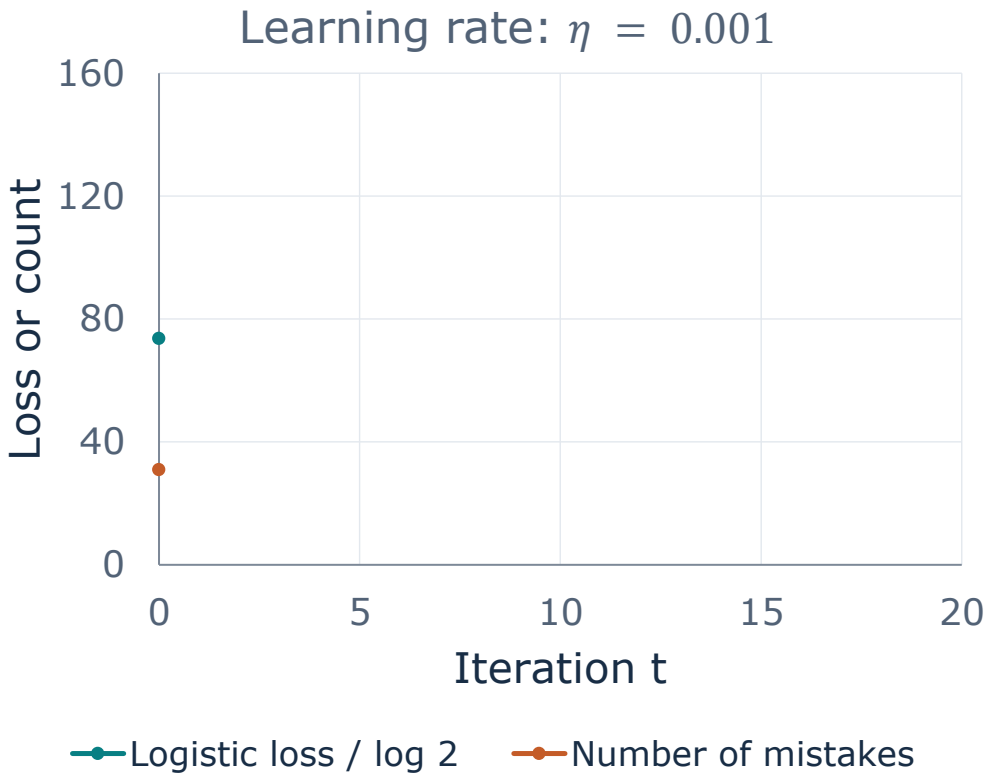
Warning 1: decreasing the empirical logistic loss
does not guarantee fewer mistakes (a lower empirical 0-1 loss).

Warning 2: gradient descent still needs an appropriate learning rate.
A step that is too large can increase the loss.

Gradient descent: $\eta = 0.001, t = 0$

Initial coefficients: $b^{(0)} = -5.75,$ $w_1^{(0)} = 0,$ $w_2^{(0)} = 1$



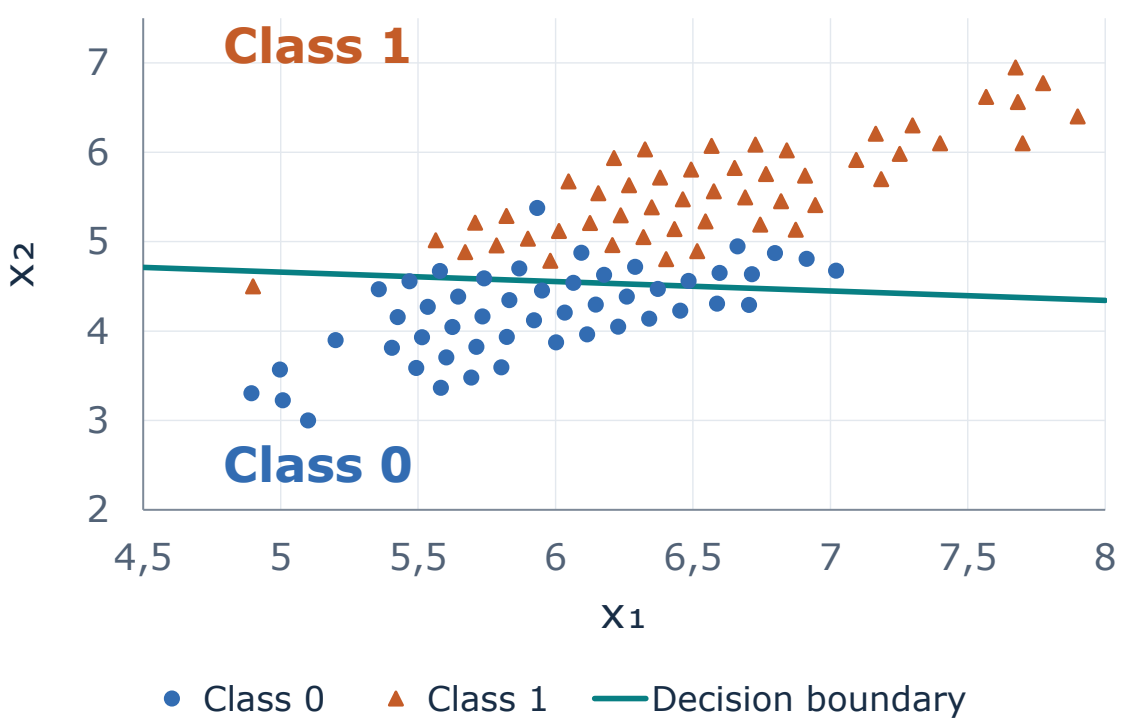


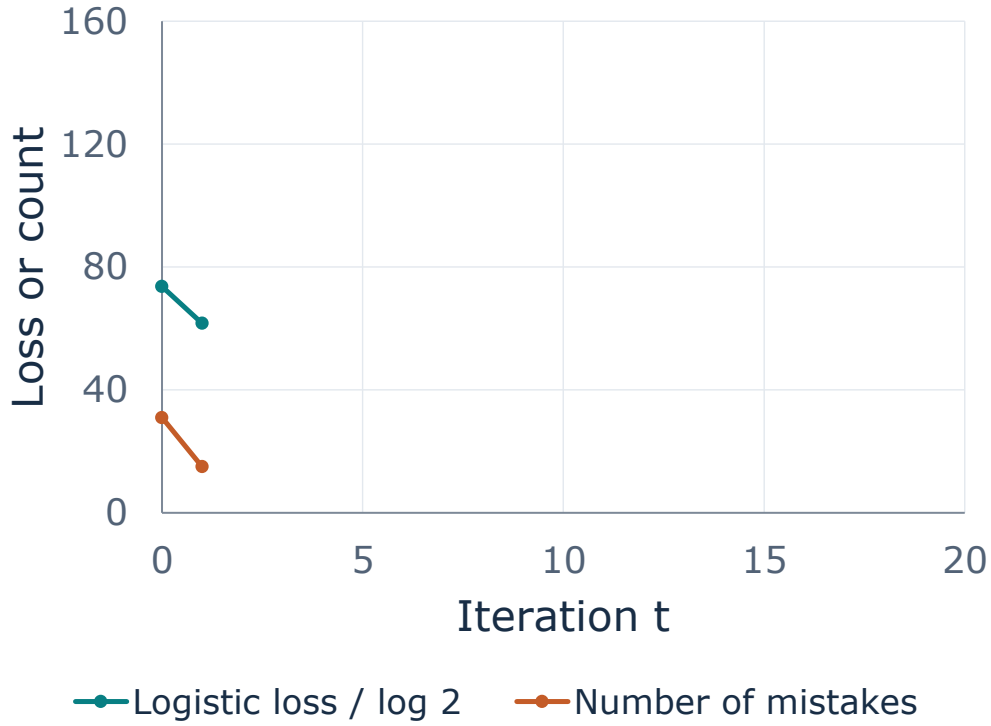
$$x_2 = 5.7500 + 0.0000x_1$$

Logistic loss: 51.048
Mistakes: 31 of 100

Gradient descent: $\eta = 0.001, t = 1$

Coefficients: $b^{(1)} \approx -5.732582$ $w_1^{(1)} \approx 0.116619$ $w_2^{(1)} \approx 1.105206$



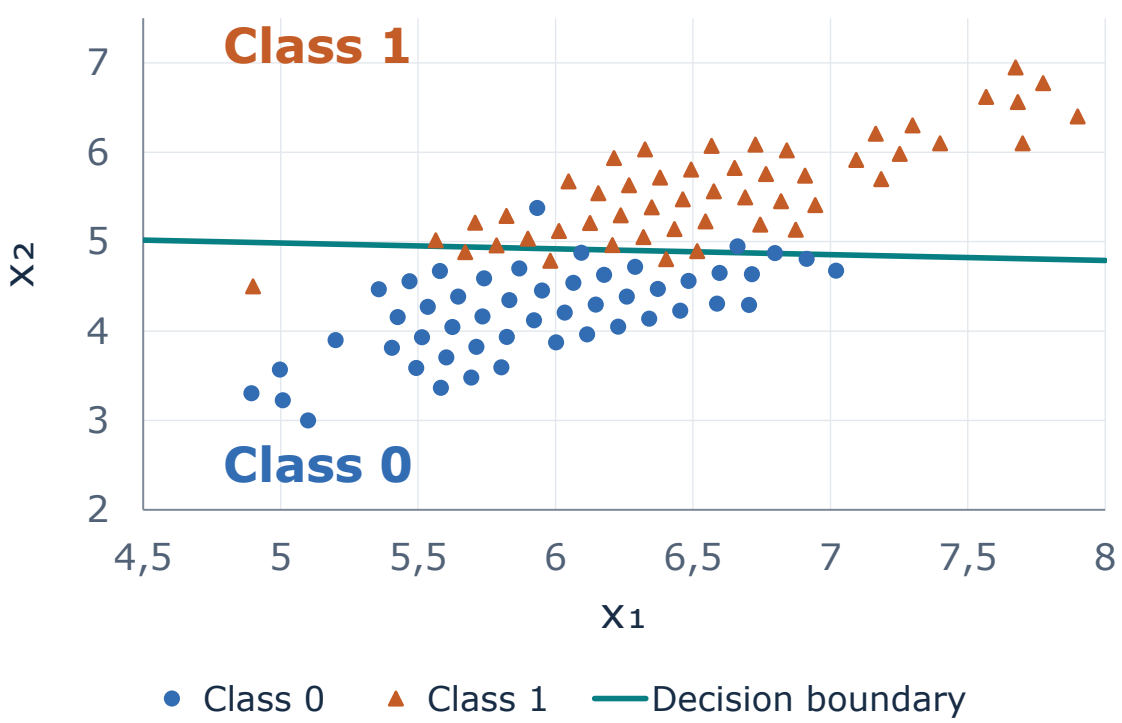


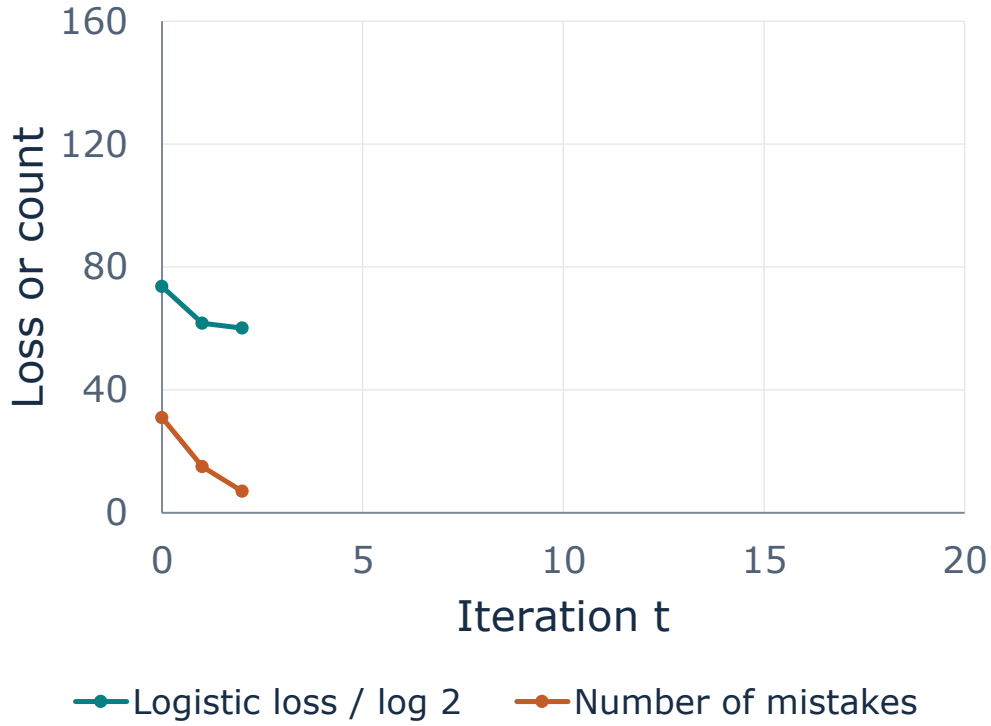
$$x_2 = 5.1869 - 0.1055x_1$$

Logistic loss: 42.733
Mistakes: 15 of 100

Gradient descent: $\eta = 0.001, t = 2$

Coefficients: $b^{(2)} \approx -5.740878$ $w_1^{(2)} \approx 0.070300$ $w_2^{(2)} \approx 1.081132$



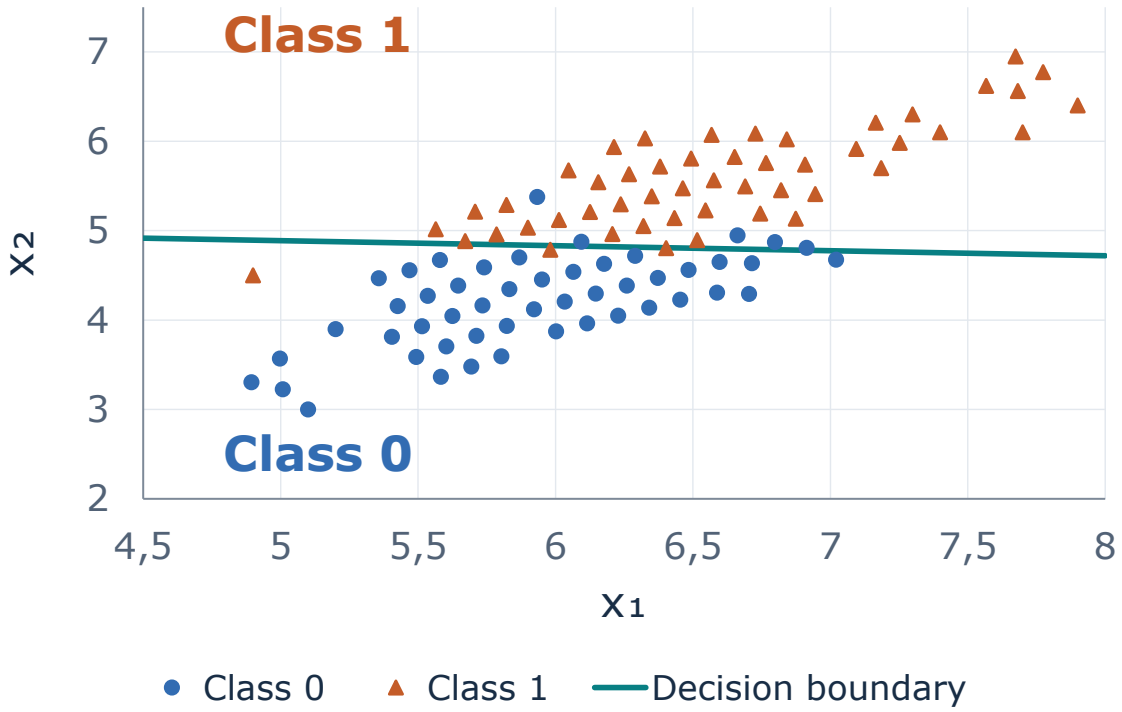


$$x_2 = 5.3101 - 0.0650x_1$$

Logistic loss: 41.668
Mistakes: 7 of 100

Gradient descent: $\eta = 0.001, t = 5$

Coefficients: $b^{(5)} \approx -5.744795$ $w_1^{(5)} \approx 0.062560$ $w_2^{(5)} \approx 1.111266$



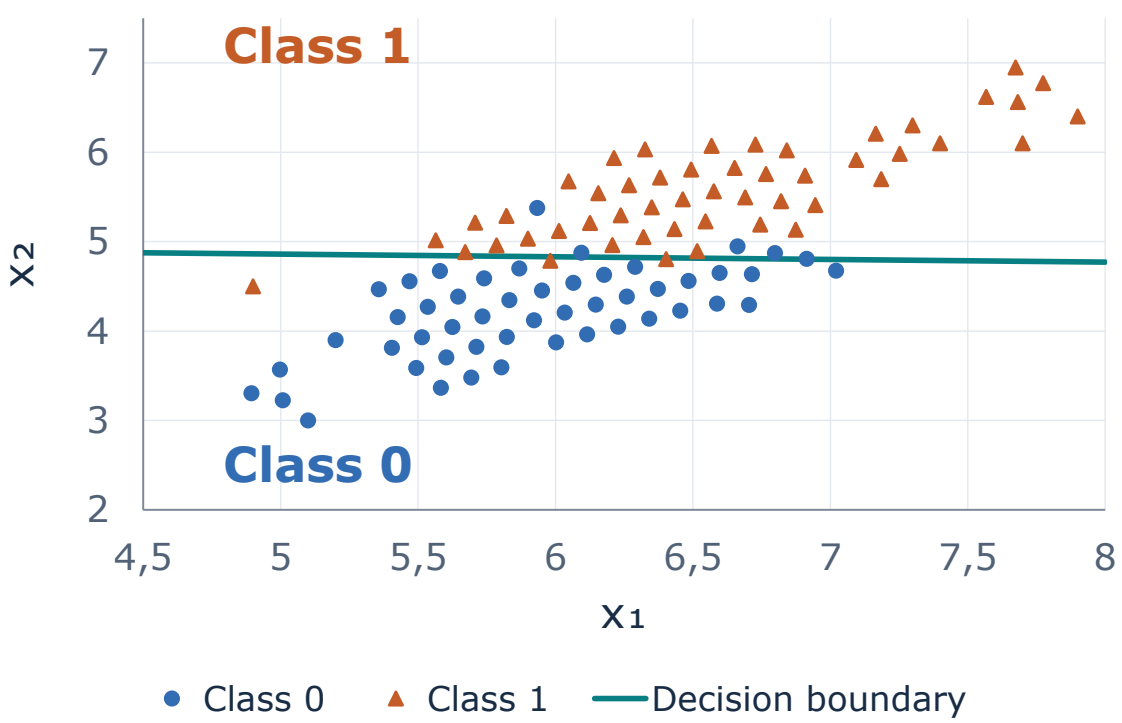


$$x_2 = 5.1696 - 0.0563x_1$$

Logistic loss: 41.300
Mistakes: 8 of 100

Gradient descent: $\eta = 0.001, t = 10$

Coefficients: $b^{(10)} \approx -5.753743$ $w_1^{(10)} \approx 0.034272$ $w_2^{(10)} \approx 1.148470$



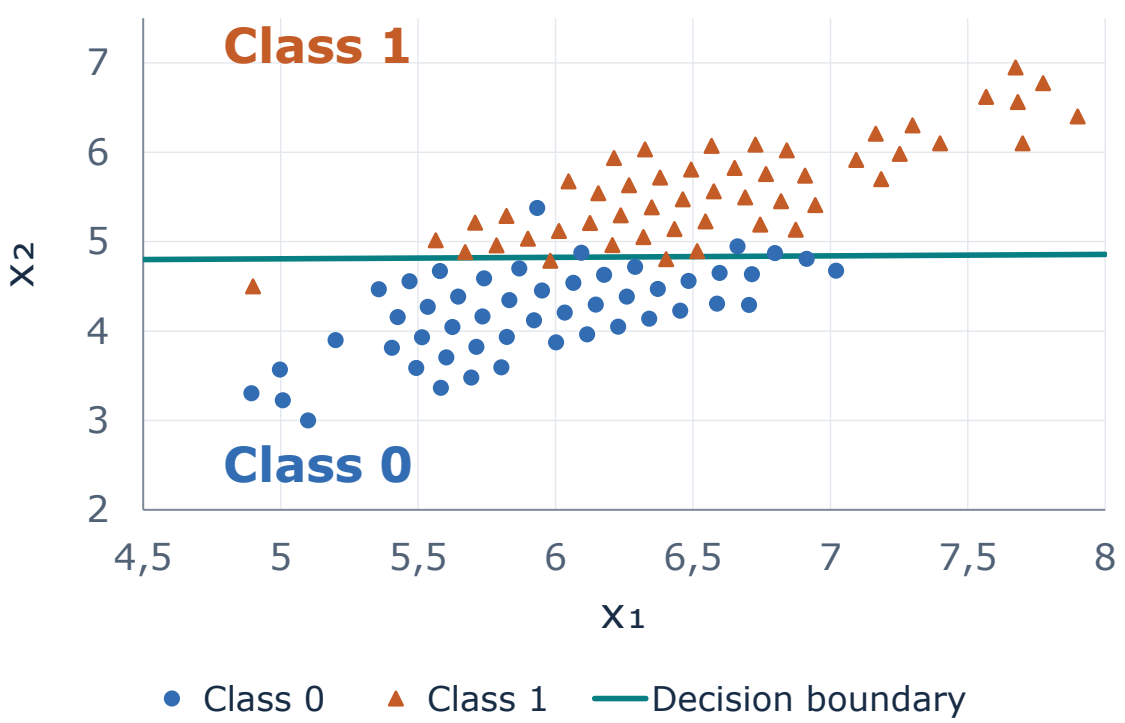


$$x_2 = 5.0099 - 0.0298x_1$$

Logistic loss: 40.848
Mistakes: 8 of 100

Gradient descent: $\eta = 0.001, t = 20$

Coefficients: $b^{(20)} \approx -5.771210$ $w_1^{(20)} \approx -0.020164$ $w_2^{(20)} \approx 1.221302$





$$x_2 = 4.7255 + 0.0165x_1$$

Logistic loss: 39.992
Mistakes: 7 of 100

The loss can fall while mistakes increase

The same planar run shows the limitation of the surrogate.



Step	Logistic loss	Errors
3	41.4948	7
4	41.3932	8

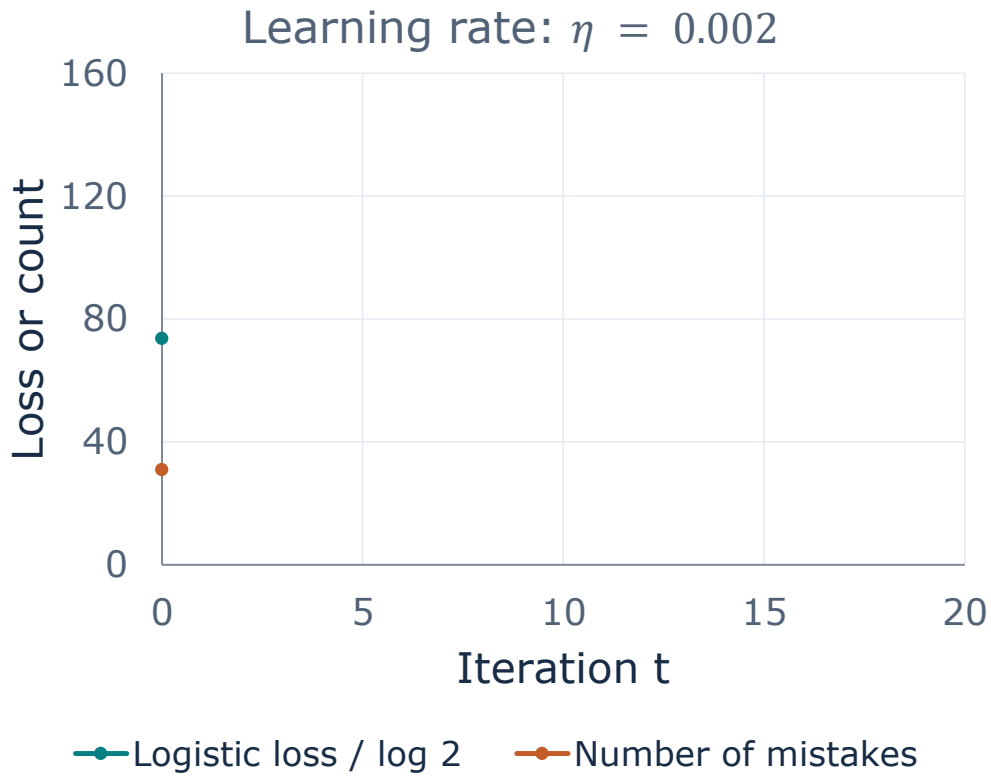
Of course: the bound still holds at both parameter choices.

Logistic loss can decrease even when the mistake count increases.

Gradient descent: $\eta = 0.002, t = 0$

Initial coefficients: $b^{(0)} = -5.75,$ $w_1^{(0)} = 0,$ $w_2^{(0)} = 1$



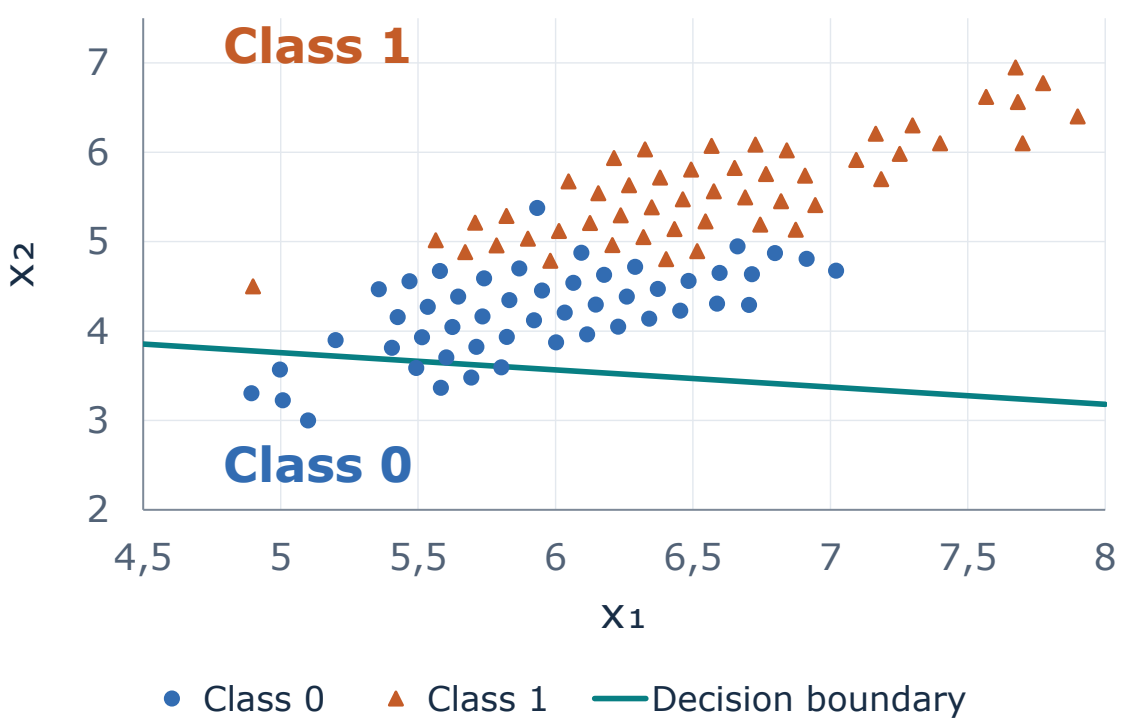


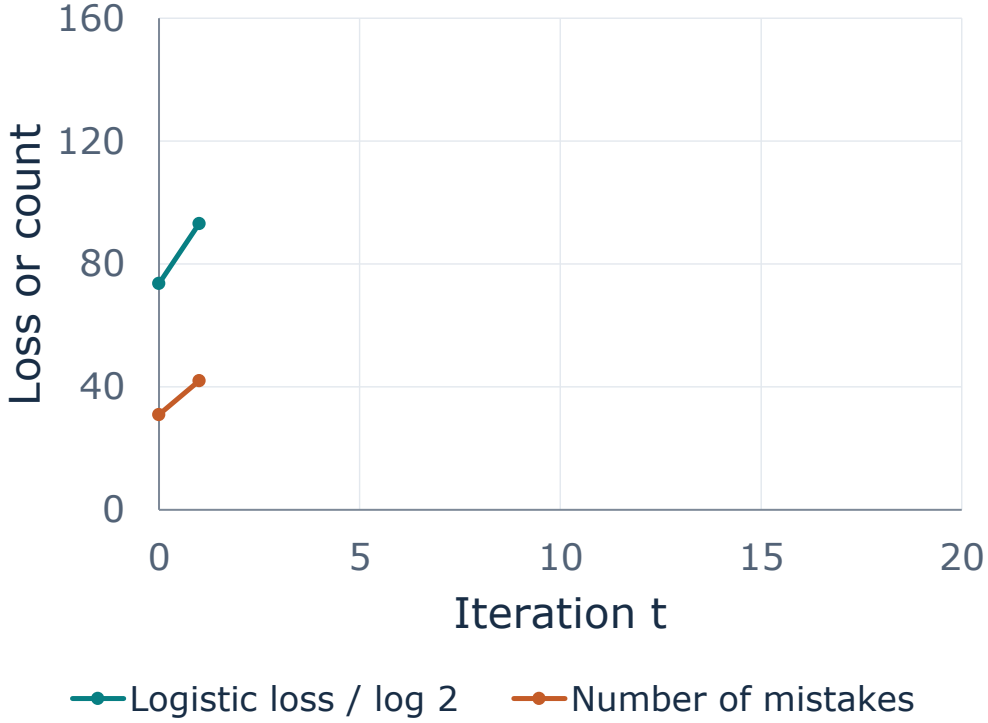
$$x_2 = 5.7500 + 0.0000x_1$$

Logistic loss: 51.048
Mistakes: 31 of 100

Gradient descent: $\eta = 0.002, t = 1$

Coefficients: $b^{(1)} \approx -5.715164$ $w_1^{(1)} \approx 0.233238$ $w_2^{(1)} \approx 1.210411$



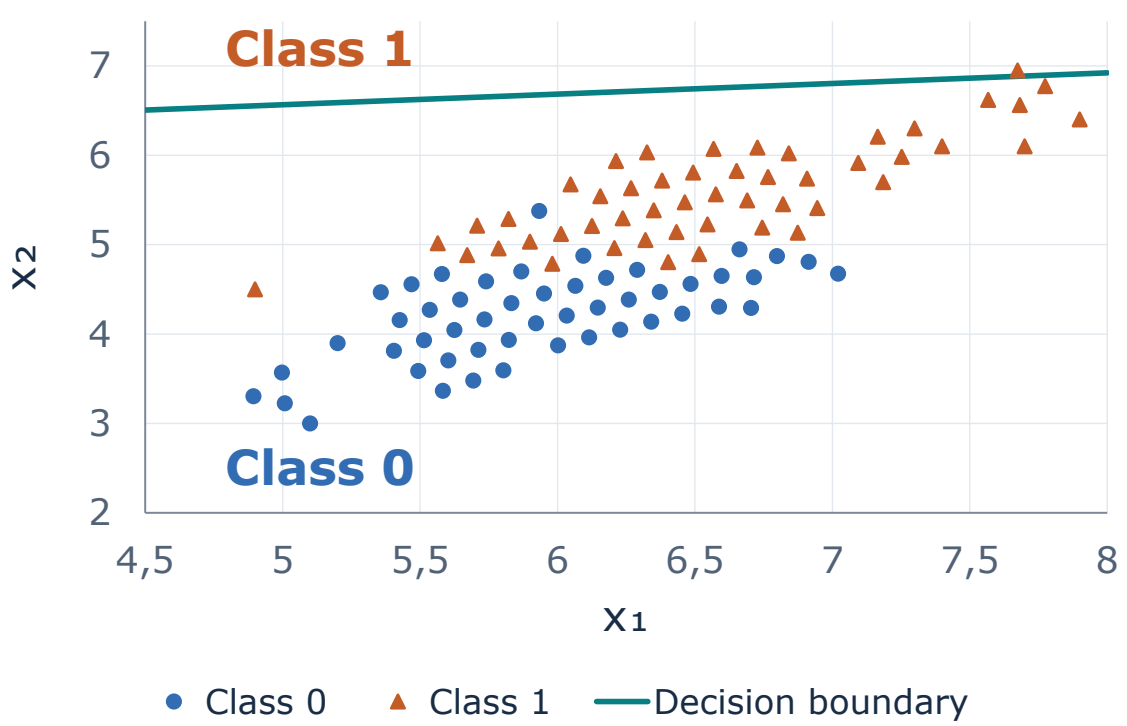


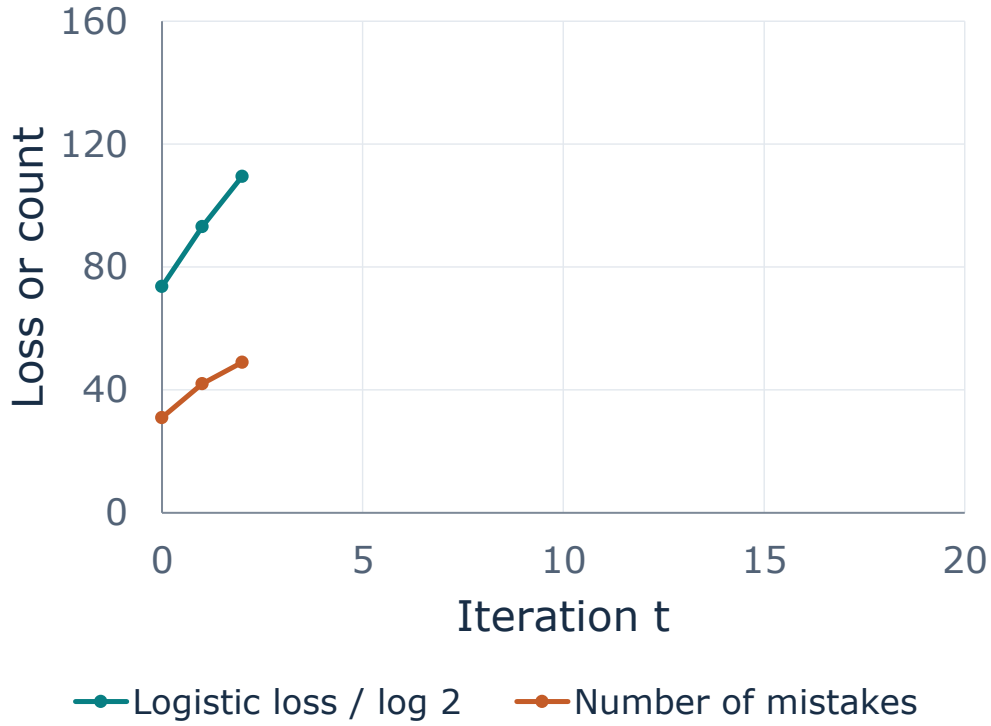
$$x_2 = 4.7217 - 0.1927 x_1$$

Logistic loss: 64.556
Mistakes: 42 of 100

Gradient descent: $\eta = 0.002, t = 2$

Coefficients: $b^{(2)} \approx -5.773416$ $w_1^{(2)} \approx -0.115049$ $w_2^{(2)} \approx 0.966988$



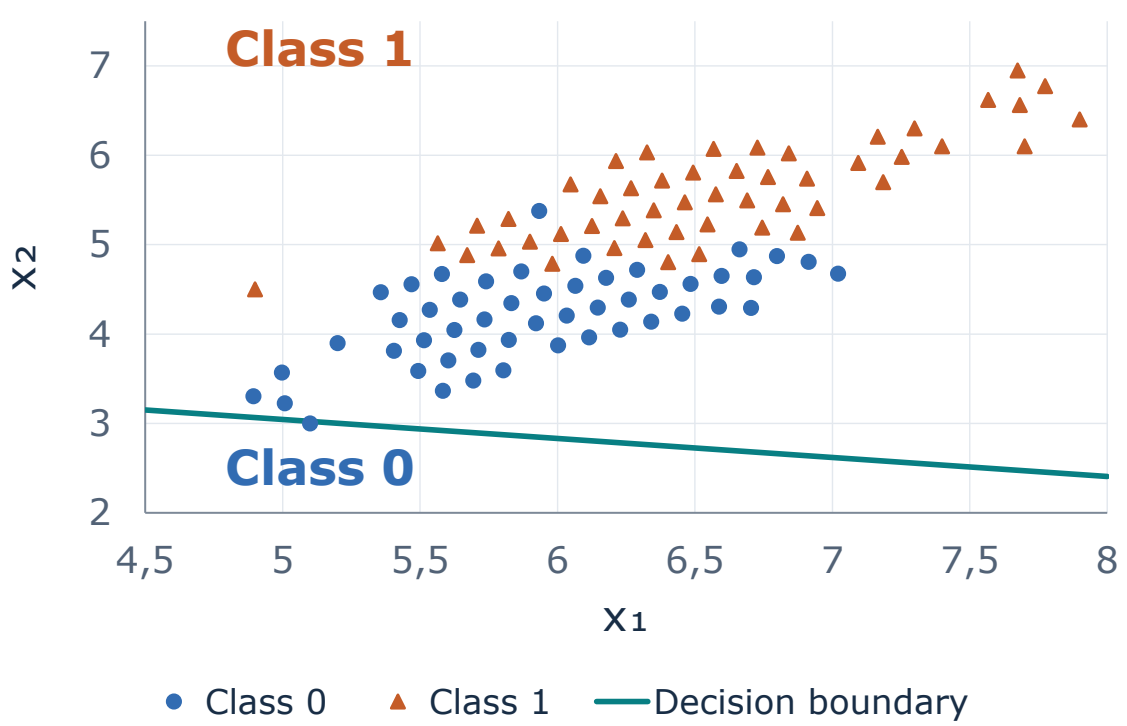


$$x_2 = 5.9705 + 0.1190x_1$$

Logistic loss: 75.928
Mistakes: 49 of 100

Gradient descent: $\eta = 0.002, t = 5$

Coefficients: $b^{(5)} \approx -5.718821$ $w_1^{(5)} \approx 0.295982$ $w_2^{(5)} \approx 1.392324$



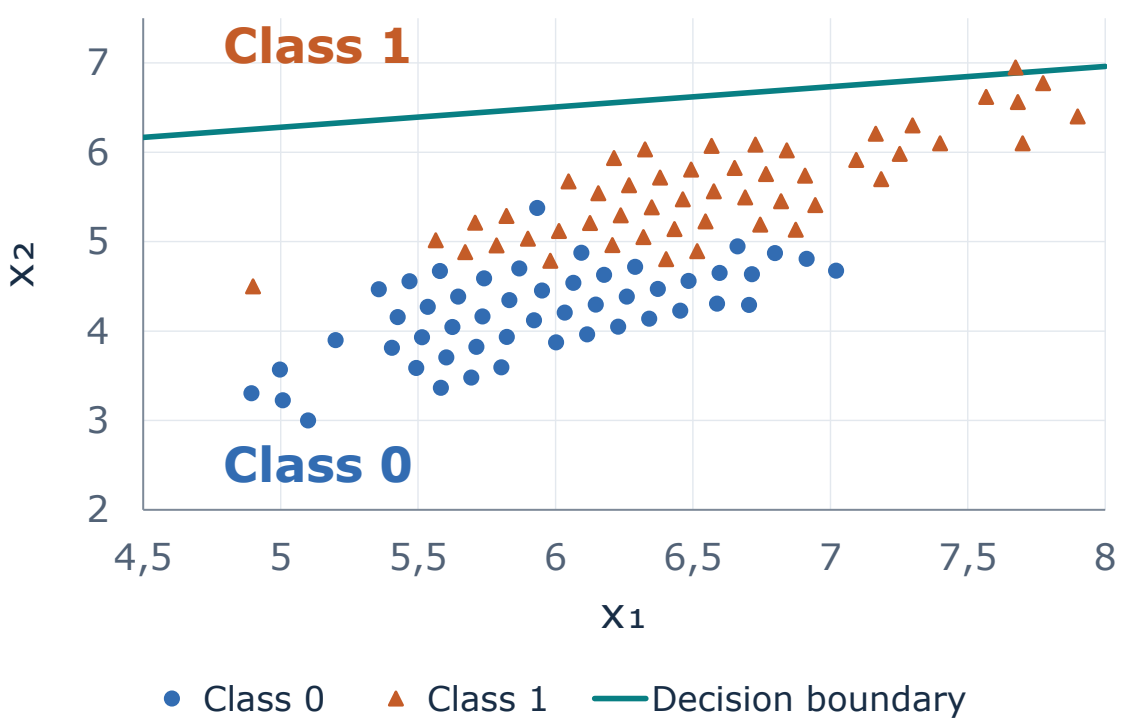


$$x_2 = 4.1074 - 0.2126x_1$$

Logistic loss: 105.343
Mistakes: 49 of 100

Gradient descent: $\eta = 0.002, t = 10$

Coefficients: $b^{(10)} \approx -5.825631$ $w_1^{(10)} \approx -0.256765$ $w_2^{(10)} \approx 1.132075$



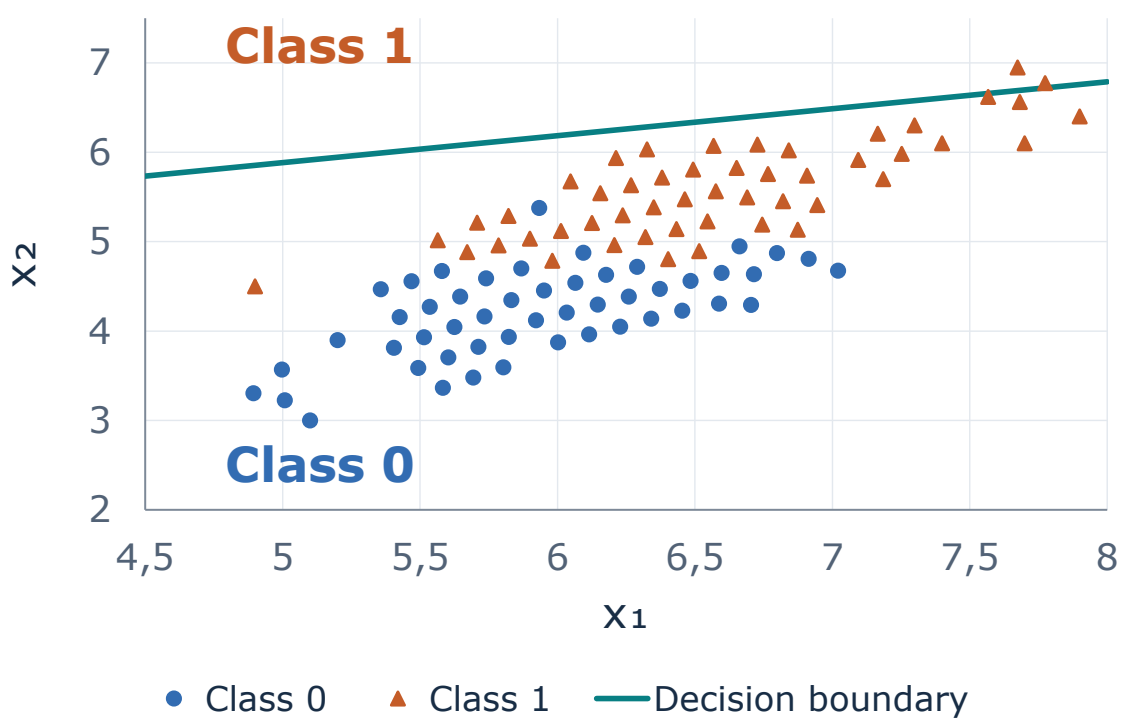


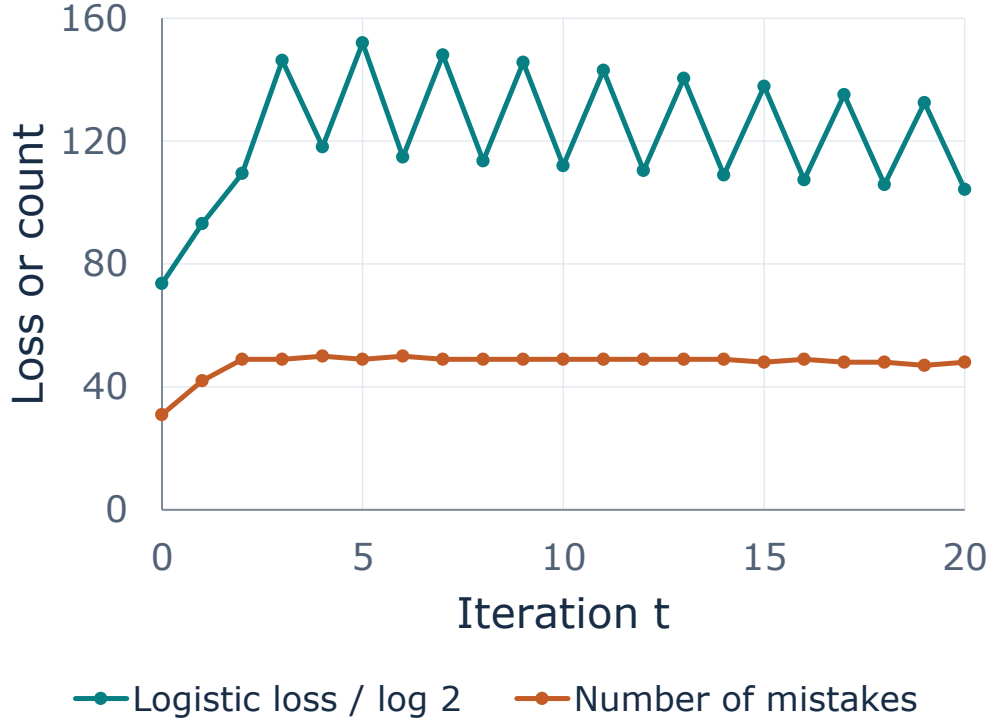
$$x_2 = 5.1460 + 0.2268x_1$$

Logistic loss: 77.632
Mistakes: 49 of 100

Gradient descent: $\eta = 0.002, t = 20$

Coefficients: $b^{(20)} \approx -5.884803$ $w_1^{(20)} \approx -0.405317$ $w_2^{(20)} \approx 1.344522$



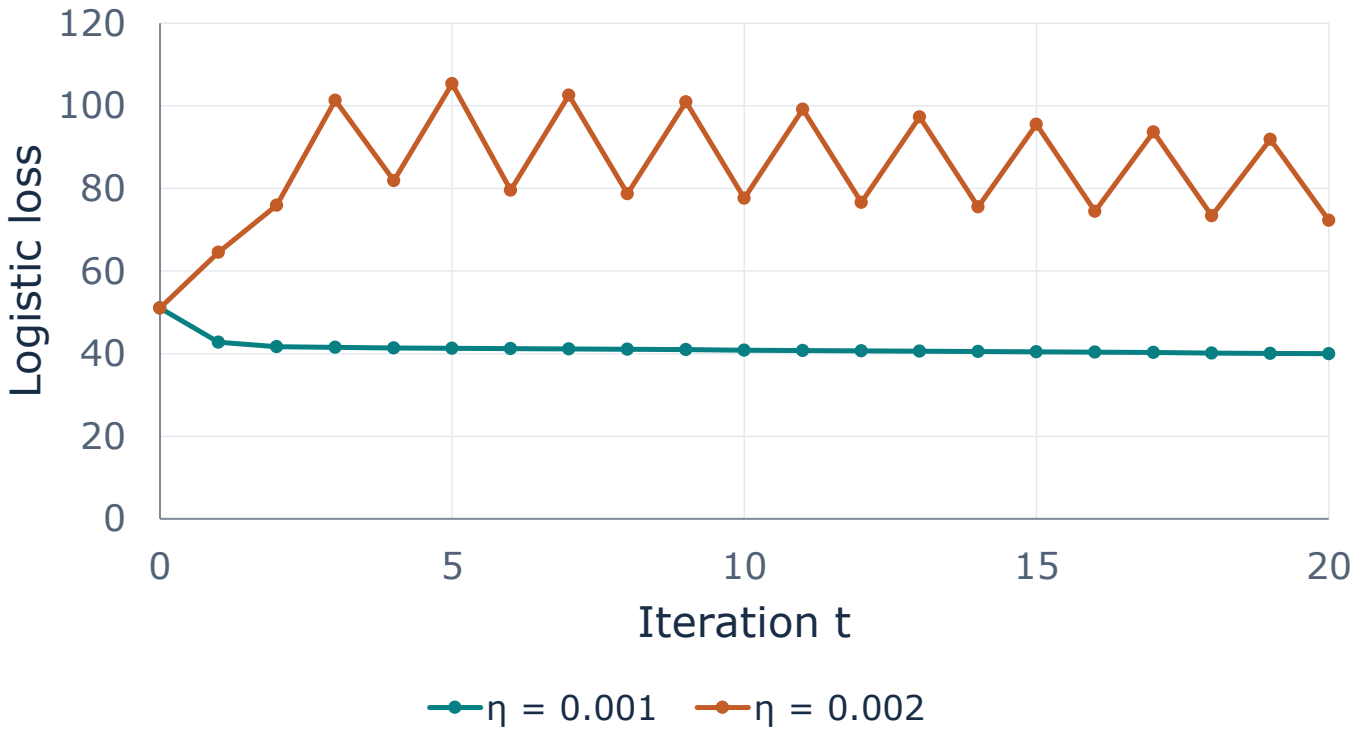


$$x_2 = 4.3769 + 0.3015x_1$$

Logistic loss: 72.282
Mistakes: 48 of 100

Step size and logistic loss

Same data and initial coefficients, only the learning rate changes.



Step	Loss $\eta = 0.001$	Loss $\eta = 0.002$
0	51.048	51.048
1	42.733	64.556

At $\eta = 0.002$, even the first update increases the loss.

A step that is too large can increase the loss.

Today's lecture

Binary classification

Classification error and logistic loss

Gradient descent

More than two classes

More than two classes

Recognising a handwritten digit means choosing among ten classes: $0, 1, \ldots, 9$.



Each row contains examples of the same digit.

An image is made of pixels.
In greyscale, their intensities form a matrix of real numbers.

We read the matrix row by row to form a row vector.
We use the same pixel order for every image.

$$\mathbf{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$$

Here, d is the number of pixels.

We predict one probability per class

Predicting class probabilities

We number the classes from 1 to K.
For digits 0 to 9, class index k refers to digit $k - 1$.

$$\mathbf{p} : \mathbb{R}^d \longrightarrow [0,1]^K$$

$$\mathbf{p}(\mathbf{x}) = \big(p_1(\mathbf{x}), \ldots, p_K(\mathbf{x})\big)$$

Each component gives the predicted probability of the corresponding class.
For this vector to represent a valid probability distribution,
we require its components to sum to one:

$$\sum_{k=1}^{K} p_k(\mathbf{x}) = 1$$

If we must choose one label, which should we predict?

The multiclass plug-in rule

Choose the class with the largest predicted probability.

$$h_{\mathbf{p}} : \mathbb{R}^d \longrightarrow \{1, \dots, K\}$$

$h_{\mathbf{p}}(\mathbf{x})$ returns the index of the class $k \in \{1, \dots, K\}$
with the largest predicted probability $p_k(\mathbf{x})$.

In case of a tie, we return
the smallest index among those having the largest predicted probability.

How do we build the probability function?

One affine score per class

for each class $k = 1, \dots, K$

$$b_k \in \mathbb{R}, \qquad \mathbf{w}_k = \begin{pmatrix} w_{1k} \\ \vdots \\ w_{dk} \end{pmatrix} \in \mathbb{R}^{d \times 1} \qquad \boldsymbol{\theta}_k = \begin{pmatrix} b_k \\ \mathbf{w}_k \end{pmatrix} \in \mathbb{R}^{(d+1) \times 1}$$

The k-th score function is defined via

$$f_{\boldsymbol{\theta}_k}(\mathbf{x}) = b_k + \mathbf{x}\mathbf{w}_k$$

As in binary classification, these affine scores are also called logits.

Stack all the parameter columns in a parameter matrix Θ

$$\boldsymbol{\Theta} = (\boldsymbol{\theta}_1 \quad \cdots \quad \boldsymbol{\theta}_K) \in \mathbb{R}^{(d+1) \times K}$$

$$\mathbf{f}_{\boldsymbol{\Theta}}(\mathbf{x}) = \left(f_{\boldsymbol{\theta}_1}(\mathbf{x}), \dots, f_{\boldsymbol{\theta}_K}(\mathbf{x}) \right) \in \mathbb{R}^K$$

Softmax turns scores into probabilities

Warning: separate sigmoids do not ensure a sum of one!!!

Three zero scores would give three probabilities of one half.

Exponentiate each score, then divide by the sum across classes.

$$\text{softmax} : \mathbb{R}^K \longrightarrow (0,1)^K$$

$$\mathbf{s} = (s_1, \ldots, s_K) \in \mathbb{R}^K$$

$$\text{softmax}(\mathbf{s}) = \left(\frac{e^{s_1}}{\sum_{r=1}^{K} e^{s_r}}, \ldots, \frac{e^{s_K}}{\sum_{r=1}^{K} e^{s_r}} \right)$$

Positive numbers: $[\text{softmax}(\mathbf{s})]_k > 0$

Valid probabilities: $\sum_{k=1}^{K} [\text{softmax}(\mathbf{s})]_k = \frac{\sum_{k=1}^{K} e^{s_k}}{\sum_{r=1}^{K} e^{s_r}} = 1$

Our multiclass probability model

Apply softmax to the row of affine scores.

$$\mathbf{p}_{\boldsymbol{\Theta}}(\mathbf{x}) = \operatorname{softmax}\big(\mathbf{f}_{\boldsymbol{\Theta}}(\mathbf{x})\big)$$

Its component for each class is

$$p_{\boldsymbol{\Theta},k}(\mathbf{x}) \coloneqq [\mathbf{p}_{\boldsymbol{\Theta}}(\mathbf{x})]_k = \frac{\exp\big(f_{\boldsymbol{\theta}_k}(\mathbf{x})\big)}{\sum_{r=1}^{K} \exp\big(f_{\boldsymbol{\theta}_r}(\mathbf{x})\big)}, \qquad k = 1, \ldots, K$$

Once we have the probability predictor $\mathbf{p}_{\boldsymbol{\Theta}}$

We can build the plug-in classifier $h_{\mathbf{p}_{\boldsymbol{\Theta}}}$

Cross-entropy loss for the observed class

The loss is minus the logarithm of the predicted probability of the observed class

$$\mathbf{p} = (p_1, \dots, p_K), \qquad y \in \{1, \dots, K\}, \qquad p_y > 0$$

$$\ell_{\text{multi}}(\mathbf{p}, y) = -\log(p_y)$$

Dataset: $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N) \in \mathbb{R}^d \times \{1, \dots, K\}$

Empirical cross-entropy

$$\mathcal{L}_{\text{multi}}(\boldsymbol{\Theta}) = \sum_{i=1}^{N} \ell_{\text{multi}}(\mathbf{p}_{\boldsymbol{\Theta}}(\mathbf{x}_i), y_i)$$

Fit the parameter matrix by minimising this summed loss, e.g., via gradient descent.

$$\text{number of mistakes made by } h_{\mathbf{p}_{\boldsymbol{\Theta}}} \leq \frac{1}{\log(2)} \mathcal{L}_{\text{multi}}(\boldsymbol{\Theta})$$

Lecture notes and exercises

The notes for today's lecture are available on the course GitHub page.

They explain the material in more detail and include exercises.

I will post the exercise solutions before the next lecture.

github.com/semtm0051/2026_27

Scan for course materials